

Background: Duke Kunshan University (DKU) is an interdisciplinary institution that grants dual undergraduate degrees, an MOE Chinese degree and a degree from Duke University in Durham, United States. The principal structure of DKU majors is robustly interdisciplinary. No student confines their study to a single discipline (for example, biology or economics). Instead, all students engage in broad inquiry related to a subject or question (for example, political economy or global health) and take a wide variety of courses related to that area (for example, in public policy, history, ethics, or economics). As a result, our graduates are prepared to engage in a wide variety of inquiries using multiple methodologies to address complex issues that require interdisciplinary approaches. This has implications for our vision and expectations of undergraduate theses and design projects, which reflect this broad interdisciplinary training. At DKU, every student completes a two-year project known as signature work which consists of multiple interconnected parts including thematic courses, experiential learning, capstones, and a final product. It seeks to integrate students' interdisciplinary educational experience and culminates in the creation of a product in a scholarly, creative, or applied nature in lieu of an undergraduate thesis or design required by JED. Because DKU encourages students to cultivate their independence and creativity as one of its institutional student learning outcomes, the student-led signature work projects often reflect students' own particular interdisciplinary interests and training. In addition, signature work has an intensive emphasis on problem-solving and skill-development which is much needed for any interdisciplinary inquiry; thus, students' final products are evidence of transferrable skills that students have acquired and demonstrated through the 2-year program, rather than content knowledge narrowly defined by disciplinary training. In sum, while the Chinese major declared with any given student might be construed narrowly, the experience of our students is much broader—and intentionally so. This is a distinctive feature of our curriculum, and this distinctiveness results in broadly interdisciplinary submissions from our graduates' submitting theses or design projects. We have designed this to prepare our students for a wide variety of graduate programs in China and the West, where interdisciplinary training is a competitive advantage.

DIFF-EVAL: AN AI-DRIVEN AGENTIC WORKFLOW FOR CODE QUALITY EVALUATION

by

Jiahe (Jay) Chen

Class of 2026

Computation and Design (Computer Science)

Signature Work Product, in partial fulfillment of the
Duke Kunshan University Undergraduate Degree Program

May 2026

Signature Work Program

Duke Kunshan University

APPROVALS

Mentor: Jiacheng Shen, Assistant Professor of Electrical and Computer Engineering, Division
of Natural and Applied Sciences

CONTENTS

Acknowledgements	iii
Abstract	iv
List of Figures	iv
List of Tables	vi
1 Introduction	1
2 Background	3
2.1 SOLID Principles	3
2.2 Traditional Code Quality Evaluation	4
2.3 LLM-Based SOLID Violation Detection	4
2.4 Agentic Approaches to Code Quality	5
3 Methodology	7
3.1 The Diff Eval Workflow	7
3.2 Technical Implementation	9
3.3 Baseline Methods	10
3.4 Benchmark Dataset	10
3.5 Evaluation Metrics	11
3.6 Experimental Setup	12

4	Results	13
4.1	Overall Performance	13
4.2	Accuracy by SOLID Principle	13
4.3	Accuracy by Difficulty Level	15
4.4	Comparison with Full-Size LLM	16
5	Discussion	17
5.1	Error Analysis	17
5.2	Why Diff Eval Works	17
5.3	Privacy and Observability	18
5.4	Limitations	19
6	Conclusions	20
	References	21

ACKNOWLEDGEMENTS

Doing signature work was a bittersweet journey, mixed with excitement and disappointment. I remember the sadness when finding out the experiment results didn't improve by even a single percent after spending a whole night working on it. But I also remember the surprise when new methods finally worked out and broke our best accuracy records.

During this period of time, the encouragement of my mentor, Jiacheng Shen, was like a silver light that motivated me to keep going. Looking back, I cannot imagine how I could have gotten here without his support. He provided topic inspiration, research methodology guidance, execution priorities, and networks to help this experiment succeed. Jiacheng had regular meetings with me to make sure the experiment was on the right track. When there were urgent issues to deal with, he even spent his personal time to help. I sincerely appreciate his time and dedication to this project. The outcome of this project is not only a paper or a diploma, but also lifelong gains in my critical thinking, academic literacy, and information retrieval ability. Your advising helped me a lot!

I would also like to express my gratitude to my parents, who financially supported me in attending DKU. Without their sponsorship, I would not even have had the opportunity to start this journey.

Thanks again to Jiacheng and my parents.

ABSTRACT

Code quality is a critical concern in software engineering, directly affecting system stability, maintainability, and development efficiency. Traditional static analysis tools detect only syntactic and structural issues through rule-based pattern matching, falling short on design-level defects such as SOLID violations that require understanding architectural intent. LLM-based analysis offers richer semantic understanding, but sending proprietary codebases to cloud APIs introduces data leakage and compliance risks that are unacceptable in many enterprise and government settings. This work proposes that, in the context of code quality evaluation, **Small LLM + Agentic Workflow** $\approx$ **Full-Size LLM**—delivering near-cloud accuracy entirely on local hardware with zero data egress.

This work proposes **Diff Eval**, a workflow-driven approach that simulates real software development scenarios—attempting to extend the code under test—to assess extensibility and surface design defects through the resulting code diff.

Evaluated on a benchmark dataset of 240 code examples (48 per principle, spanning Python, Java, Kotlin, and C#, across three difficulty levels), Diff Eval achieves **87.92%** accuracy using a local Qwen3.5 9B model via Ollama—significantly outperforming single-agent (70.00%) and two-agent (55.83%) baselines, closing approximately 70% of the performance gap to a full-size cloud LLM (95.83%), and running entirely on-premise with zero data egress. Accuracy remains robust across code complexity levels, enabled by a context-compression mechanism that insulates the system from long-code noise.

Keywords: Software engineering, LLM, code quality, design principles, agentic workflow

LIST OF FIGURES

2.1	Results from Pehlivan et al. [12] on the 240-example SOLID benchmark.	5
2.2	One-, two-, and four-agent workflow configurations for test smell detection from Melo et al. [8]. The Detector–Evaluator pattern informs our Two-Agent baseline.	6
3.1	Diff Eval system architecture: five parallel SOLID-principle channels (Scenario Generation → Code Modification → Diff Analysis → Pre-Verification) aggregated by Unified Ranking.	8
4.1	Overall accuracy comparison across the three workflow methods.	14
4.2	Accuracy comparison by SOLID principle across the three methods.	14
4.3	Top-2 accuracy comparison by difficulty level across the three methods.	15
5.1	Top-2 resolved confusion matrix for Diff Eval. Dominant error: SRP over-prediction (19 off-diagonal cases).	18

LIST OF TABLES

3.1	Principle-specific scenarios and diff signals used in Stage 1 of Diff Eval.	9
4.1	Overall performance comparison across three workflows on 240 examples (Qwen3.5 9B).	13
4.2	Top-2 accuracy by SOLID principle (48 examples each).	14
4.3	Accuracy by difficulty level (80 examples each). Difficulty is determined by the number of lines of code.	15
4.4	Qwen3.5 9B workflows vs. Qwen3.5 Plus (cloud). Diff Eval closes $\sim 70\%$ of the performance gap through workflow design alone.	16

Chapter 1

INTRODUCTION

Software design quality is a critical concern in modern software engineering. While syntactic errors and surface-level bugs can be caught by compilers and static analyzers, *design-level* defects—violations of principles like SOLID [6, 7]—remain notoriously difficult to detect automatically. These violations degrade maintainability, increase coupling, and raise the cost of future modifications [11, 14].

Recent advances in large language models (LLMs) have opened new possibilities for code understanding [3]. However, directly prompting an LLM to judge whether a code snippet violates a design principle yields poor results—even with optimized prompt strategies, accuracy remains limited and varies dramatically across principles [12]. Meanwhile, agentic multi-model workflows have shown promise for related tasks like test smell detection [8], suggesting that workflow design matters as much as model capability.

A practical constraint motivates fully local deployment. Enterprise and government codebases routinely contain proprietary or regulated information that cannot be transmitted to external cloud APIs without risk of data leakage or compliance violation. Regulatory frameworks such as the EU AI Act [2] and national security auditing criteria such as Katakri 2020 [9] impose strict data-residency and access-control requirements on organisations handling sensitive data. Research on cloud computing privacy further highlights systemic risks of data outsourcing—including insider threats, unauthorised access, and loss of organisational control over sensitive

assets [4]. These pressures from both industry and academia motivate a code-quality analysis system that runs entirely on-premise with zero data egress.

This work proposes a fundamentally different approach grounded in a core hypothesis: **simulating code extension reveals design defects more effectively than asking a model to judge code quality directly**. When a developer tries to extend poorly designed code, the design flaws surface as forced, wide-ranging modifications to existing logic. By observing the diff that results from such a simulated extension, a system can detect design violations through objective evidence rather than subjective LLM assessment.

The **Diff Eval** workflow operationalizes this principle through five parallel detection channels, one per SOLID principle. Each channel generates a principle-specific extension scenario, modifies the code accordingly, computes a unified diff, and checks whether the diff pattern matches a known violation signal. A final ranking step aggregates results across all five channels.

The contributions of this work are:

1. A **diff-based detection methodology** that converts subtle structural violations into concrete, observable diff patterns—making them tractable for small LLMs.
2. A **context-compression mechanism** (the Pre-Verification stage) that isolates the diff signal from full source context, enabling robust detection across code of varying complexity.
3. A **fully local, privacy-preserving** system that demonstrates a core principle: **Small LLM + Agentic Workflow $\approx$ Full-Size LLM**. Diff Eval on a local 9B model achieves 87.92% accuracy, closing approximately 70% of the performance gap between a plain small-model baseline (70.00%) and a full-size cloud LLM (95.83%), while running entirely on consumer hardware with zero data egress.

Chapter 2

BACKGROUND

2.1 SOLID Principles

The SOLID principles, formulated by Robert C. Martin [7], are five design guidelines for object-oriented software:

- **SRP** (Single Responsibility Principle): A class should have only one reason to change.
- **OCP** (Open/Closed Principle): Software entities should be open for extension but closed for modification.
- **LSP** (Liskov Substitution Principle): Subtypes must be substitutable for their base types without altering program correctness.
- **ISP** (Interface Segregation Principle): Clients should not be forced to depend on interfaces they do not use.
- **DIP** (Dependency Inversion Principle): High-level modules should depend on abstractions, not concrete implementations.

Violations of these principles lead to rigid, fragile, and tightly coupled code that resists modification [1, 6].

2.2 Traditional Code Quality Evaluation

Traditional code quality evaluation relies on static analysis tools (SonarQube, Checkstyle, PMD) and metric-based approaches (CBO, RFC, LCOM [1]). Static analyzers apply rule-based pattern matching and are fast, but detect only syntactic and structural issues; design-level defects like SOLID violations require understanding *intent* and *architectural context* that rules cannot capture [14]. Metric-based approaches quantify structural properties but cannot identify *which* principle is violated or *why*. ML classifiers [11] can detect subtler patterns but require large labeled corpora and do not generalize across languages without retraining. These limitations motivate LLM-based approaches, which learn code semantics from large corpora and generalize across languages without per-language engineering.

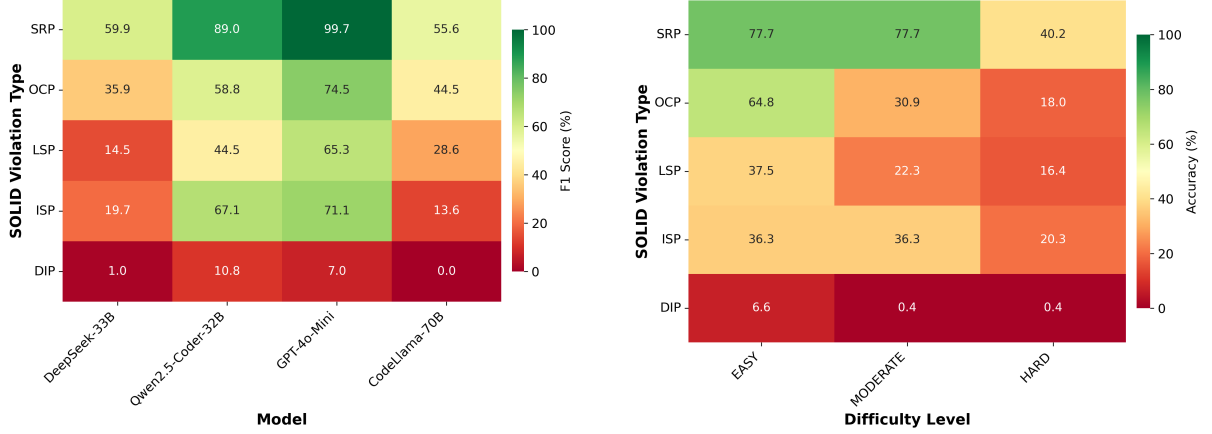
2.3 LLM-Based SOLID Violation Detection

Pehlivan et al. [12] conduct the first systematic empirical study on prompting LLMs to detect SOLID violations, evaluating CodeLlama 70B, DeepSeek-Coder 33B, Qwen2.5-Coder 32B, and GPT-4o-mini on a 240-example benchmark under four prompt strategies. The few-shot (example) strategy consistently performs best; GPT-4o-mini peaks at 69.2% and Qwen2.5-Coder 32B at 53.8%, while both open-source models stay below 25%. The Qwen family’s strong showing at fewer parameters motivates our choice of Qwen for local inference.

Violation-type breakdown. Figure 2.1a disaggregates results by principle. **DIP is essentially unsolved:** F1 ranges from 0.0 (CodeLlama 70B) to just 10.8 (Qwen2.5-Coder 32B). ISP is highly variable (13.6–71.1), while SRP is the most tractable (55.6–99.7). Model scale does not predict principle-level performance—CodeLlama 70B underperforms GPT-4o-mini on every principle despite its larger size.

Difficulty degradation. Figure 2.1b shows accuracy by difficulty level averaged across models. SRP holds at 77.7% through Moderate but drops to 40.2% at Hard; OCP degrades from 64.8% to 18.0%; DIP is near-zero at all levels (6.6% / 0.4% / 0.4%). Two classes emerge: *surface-visible*

principles (SRP, OCP) that degrade with complexity, and *structurally opaque* principles (DIP, ISP, LSP) that are unsolved regardless of difficulty. These patterns directly motivate Diff Eval: converting opaque structural violations into observable diff signals and compressing code context for robustness at Hard difficulty.



(a) Per-principle F1 by model. DIP is near-zero for all models.

(b) Per-principle accuracy by difficulty. DIP near-zero at all levels.

Figure 2.1: Results from Pehlivan et al. [12] on the 240-example SOLID benchmark.

Our **Single Agent baseline** replicates Pehlivan et al.’s `example` strategy using Qwen3.5 9B, establishing a direct comparison point.

2.4 Agentic Approaches to Code Quality

Agentic LLM workflows—multi-step pipelines where specialized agents collaborate rather than a single model acting in isolation [15]—have shown strong results for complex reasoning tasks. Frameworks like ReAct [16] and LangGraph [5] provide the infrastructure for stateful, multi-actor pipelines.

Most relevant to our design, Melo et al. [8] apply agentic workflows to *test smell* detection and refactoring using small models (3B–14B). They compare one-agent, two-agent (Detector + Evaluator), and four-agent configurations (Figure 2.2). Their key findings: Phi-4-14B with four agents performs within 5% of single-agent GPT-4o-mini, and the Detector–Evaluator feedback loop consistently improves detection quality over single-pass approaches.

Our **Two-Agent baseline** directly adopts the Detector–Evaluator pattern for SOLID detection,

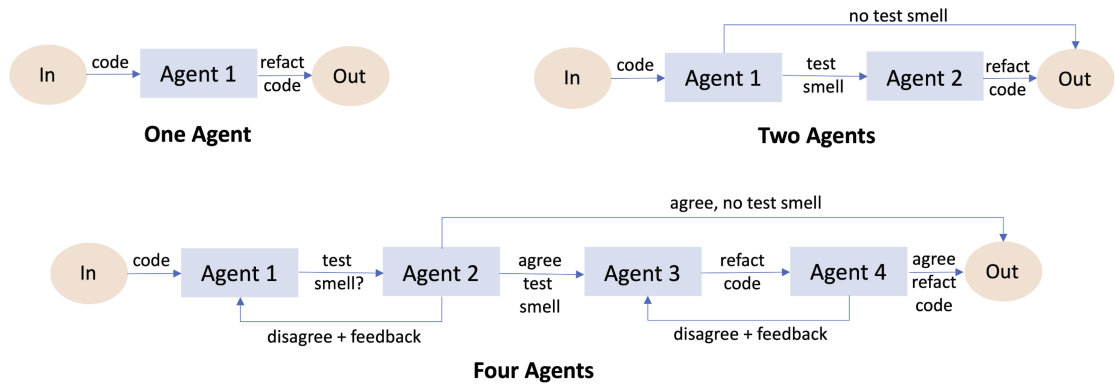


Figure 2.2: One-, two-, and four-agent workflow configurations for test smell detection from Melo et al. [8]. The Detector–Evaluator pattern informs our Two-Agent baseline.

with Agent 2 critiquing and refining Agent 1’s output over up to 3 iterations. Our **Diff Eval** workflow extends this further: rather than improving judgment through iteration, it replaces static judgment with behavioral observation—generating code diffs as objective evidence of design violations.

Chapter 3

METHODOLOGY

3.1 The Diff Eval Workflow

3.1.1 Core Insight: Code Extensibility as a Design Quality Signal

The Diff Eval workflow is grounded in a fundamental property of code extensibility: **well-designed code can be extended by adding new code, while poorly designed code forces modification of existing code.** This property generalizes across all SOLID principles: each violation type manifests as a characteristic pattern of forced modifications when the code is challenged with a principle-specific extension task.

Rather than asking an LLM to abstractly judge whether code violates a design principle, Diff Eval simulates a real development scenario: it generates a realistic feature request, asks the LLM to implement it, and then examines the resulting code diff to determine what kind of changes were required. The diff provides objective, structural evidence that is far more reliable than subjective LLM judgment.

3.1.2 Architecture Overview

The system deploys five parallel detection channels—one per SOLID principle—each operating independently through four stages, followed by a Unified Ranking step that aggregates results.

Figure 3.1 illustrates the overall architecture.

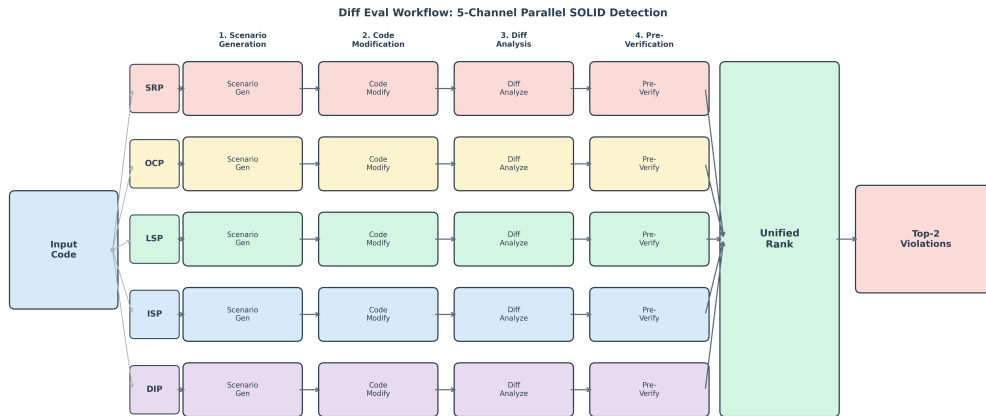


Figure 3.1: Diff Eval system architecture: five parallel SOLID-principle channels (Scenario Generation → Code Modification → Diff Analysis → Pre-Verification) aggregated by Unified Ranking.

3.1.3 Stage 1: Scenario Generation

A principle-specific prompt instructs the LLM to generate a realistic extension task tailored to surface violations of the target principle. The coupling between scenario design and signal mapping is the key innovation: each scenario is engineered so that a specific violation type produces a distinctive, recognizable diff pattern. Table 3.1 summarises the scenario and signal for each principle.

3.1.4 Stage 2: Code Modification

The LLM modifies the original code according to the generated scenario. This produces a concrete, modified version of the code that represents a realistic attempt to extend the system.

3.1.5 Stage 3: Diff Analysis

A unified diff is computed between the original and modified code. This diff is the primary evidence for violation detection—it objectively shows what changed, where changes occurred, and how extensively existing code was modified versus new code being added.

Principle	Scenario	Signal
SRP	Generate a change scoped to a single architectural layer (presentation, business, or persistence).	Modifying one layer forces edits in other layers $\Rightarrow$ mixed responsibilities.
OCF	Generate a task requiring a new type, variant, or case to be added.	New variant requires modifying existing if-else/switch logic rather than extending via new polymorphic types.
LSP	Create a polymorphic application exercising both parent and subclass through the same interface.	Subclass throws exceptions or alters return-value contracts, breaking the parent class contract.
ISP	Create a new class implementing an existing interface but needing only a subset of its methods.	New class forced to provide pass/NotImplementedError stubs for unused methods $\Rightarrow$ interface too broad.
DIP	Switch a high-level class from one concrete dependency to another without introducing abstractions.	Concrete class name changed in multiple locations inside the high-level class $\Rightarrow$ tight coupling.

Table 3.1: Principle-specific scenarios and diff signals used in Stage 1 of Diff Eval.

3.1.6 Stage 4: Pre-Verification

Each channel’s pre-verifier applies the principle-specific signal mapping to determine whether the observed diff pattern matches the expected violation signal. The pre-verifier outputs a verdict for its assigned principle, indicating whether the violation signal is present and how clearly it is observable in the diff.

3.1.7 Unified Ranking

All five pre-verification results are aggregated by a final ranking node. This node receives all five per-principle verdicts and produces a ranked list of detected violations (Top-N), ordered by how clearly the violation signal is observable in each channel’s diff.

3.2 Technical Implementation

The Diff Eval system is implemented as a LangGraph [5] state machine, where each node represents an LLM invocation or computation step and edges define the data flow between stages. LangGraph manages the five parallel detection channels and the final aggregation step

as a single directed graph with branching and merging. The LLM interface is provided by LangChain-Ollama, bridging the workflow engine to a local Ollama [10] inference server that serves quantized models with zero data egress; the primary model is Qwen3.5 9B [13]. Diffs between original and modified code are computed using Python’s `difflib` library. Each of the five channels uses principle-specific prompts for both scenario generation and pre-verification, with the full prompt templates available in the project repository.

3.3 Baseline Methods

Two baselines are used for comparison, each grounded in prior work:

- **Single Agent:** Replicates the approach of Pehlivan et al. [12]—a one-shot LLM prompt using the example (few-shot) strategy, which was shown to be the most effective prompt strategy for the Qwen model family. The LLM receives the code and directly outputs which SOLID principle is violated.
- **Two-Agent (Detector + Evaluator):** Implements the Detector–Evaluator workflow pattern from Melo et al. [8], adapted for SOLID detection. Agent 1 (Detector) identifies a potential violation with reasoning; Agent 2 (Evaluator) critiques the detection result and suggests corrections. The two agents iterate for up to 3 rounds until consensus is reached or the iteration limit is hit.

All three workflows (Single Agent, Two-Agent, Diff Eval) use the same underlying model (Qwen3.5 9B) to ensure that performance differences are attributable to workflow design rather than model capability.

3.4 Benchmark Dataset

The evaluation uses the benchmark dataset constructed by Pehlivan et al. [12], consisting of 240 manually validated code examples:

- 48 examples per SOLID principle (SRP, OCP, LSP, ISP, DIP)

- 4 programming languages: Python, Java, Kotlin, C#
- 3 difficulty levels: Easy (avg. 33 LOC), Moderate (avg. 140 LOC), Hard (avg. 306 LOC)
- Each example has a single known ground-truth violation

3.5 Evaluation Metrics

We report the following metrics:

Top-2 Accuracy. The fraction of examples where the ground-truth violation appears among the model’s top-2 ranked predictions:

$$\text{Top-2 Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[y_i \in \{\hat{y}_i^{(1)}, \hat{y}_i^{(2)}\}] \quad (3.1)$$

In the remainder of this paper, **Accuracy** refers specifically to Top-2 accuracy. Real-world code frequently exhibits multiple design violations simultaneously; Top-2 Accuracy reflects the system’s practical utility—if the correct violation appears in the top two, a developer reviewing the output will see it.

Mean Reciprocal Rank at 2 (MRR@2). Averages the reciprocal of the rank at which the ground-truth violation first appears (capped at rank 2):

$$\text{MRR@2} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i}, \quad \text{rank}_i = \begin{cases} 1 & \text{if } \hat{y}_i^{(1)} = y_i \\ 2 & \text{if } \hat{y}_i^{(2)} = y_i \\ \infty & \text{otherwise} \end{cases} \quad (3.2)$$

MRR@2 rewards systems that rank the correct answer higher: a perfect Top-1 system scores 1.0, while a system that always places the correct answer at rank 2 scores 0.5.

Macro-F1. Per-class F1 is the harmonic mean of precision and recall for each SOLID principle, where predictions are resolved from the top-2 output. Macro-F1 is the unweighted average across all five classes, treating each principle equally and penalising systems that excel on easy

principles (e.g., SRP) while failing on harder ones (e.g., DIP):

$$F_1^{(c)} = \frac{2 \cdot \text{Precision}^{(c)} \cdot \text{Recall}^{(c)}}{\text{Precision}^{(c)} + \text{Recall}^{(c)}}, \quad \text{Macro-F1} = \frac{1}{5} \sum_{c \in \{\text{SRP, OCP, LSP, ISP, DIP}\}} F_1^{(c)} \quad (3.3)$$

3.6 Experimental Setup

All experiments run locally on a consumer-grade desktop: **Intel Core i5-12400F** CPU with an **NVIDIA RTX 4060 8GB** GPU, using Ollama [10] for local inference. The primary model is Qwen3.5 9B [13] with default quantization (Q4_K_M). No data leaves the machine during inference—all 240 examples are processed entirely on local hardware.

Chapter 4

RESULTS

4.1 Overall Performance

Table 4.1 presents the overall accuracy of the three workflows on the 240-example benchmark.

Method	Accuracy	MRR@2	Macro-F1
Single Agent	70.00%	0.638	0.699
Two-Agent	55.83%	0.490	0.576
Diff Eval (ours)	87.92%	0.792	0.888

Table 4.1: Overall performance comparison across three workflows on 240 examples (Qwen3.5 9B).

Diff Eval achieves 87.92% accuracy—a 17.9 percentage-point improvement over the single-agent baseline and 32.1 pp over the two-agent approach. The MRR@2 of 0.792 (vs. 0.638 for Single Agent) indicates that when Diff Eval is correct, the ground-truth violation is ranked higher. The Macro-F1 of 0.888 (vs. 0.699 for Single Agent) confirms that the improvement is balanced across all five SOLID principles, not concentrated on a single easy class.

4.2 Accuracy by SOLID Principle

Table 4.2 breaks down accuracy by SOLID principle, revealing striking differences in where each method excels.

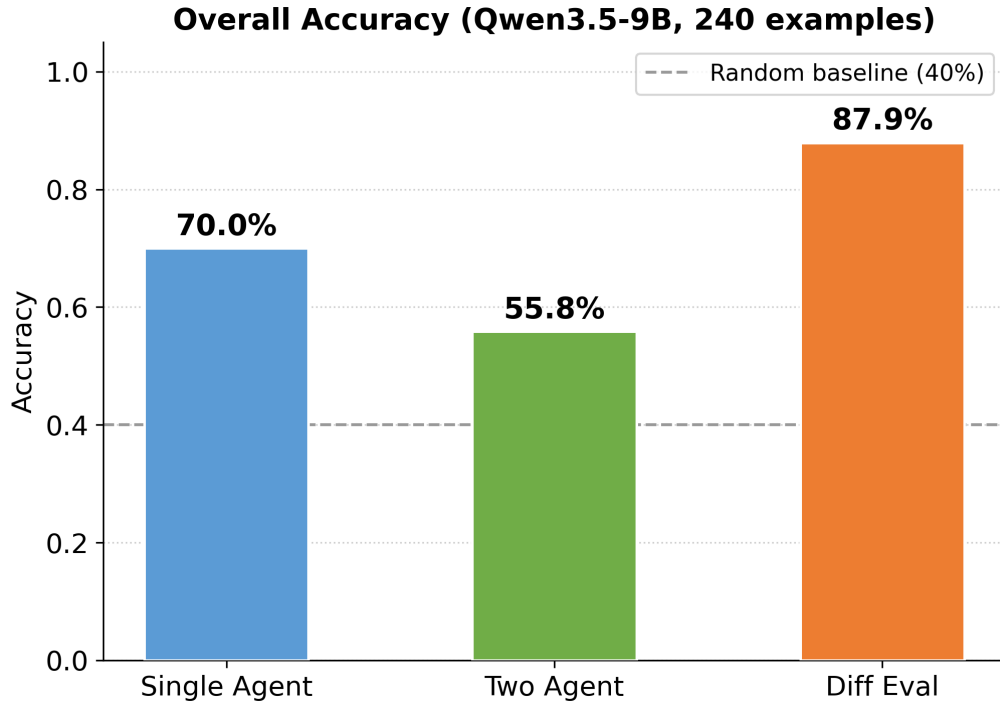


Figure 4.1: Overall accuracy comparison across the three workflow methods.

Principle	Single Agent	Two-Agent	Diff Eval
SRP	81.25%	87.50%	95.83%
OCP	97.92%	81.25%	72.92%
LSP	66.67%	33.33%	81.25%
ISP	83.33%	37.50%	97.92%
DIP	20.83%	39.58%	91.67%

Table 4.2: Top-2 accuracy by SOLID principle (48 examples each).

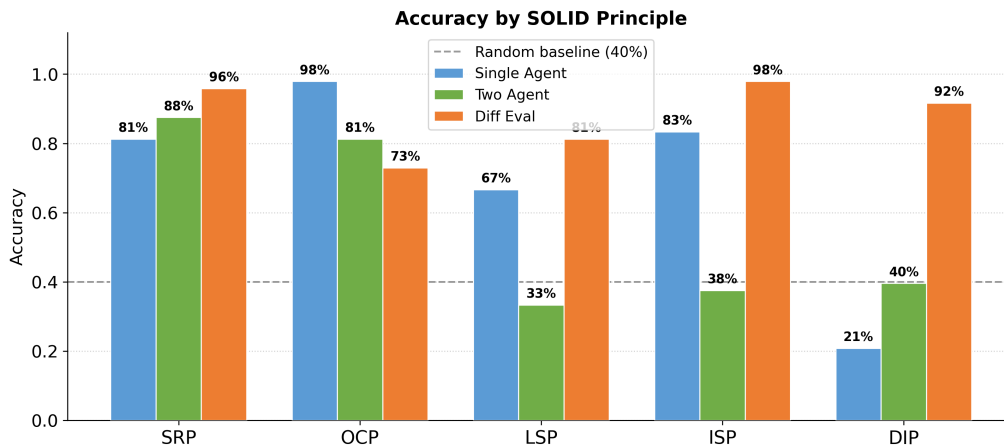


Figure 4.2: Accuracy comparison by SOLID principle across the three methods.

Diff Eval leads on four of five principles. Single Agent retains an advantage only on OCP (97.92% vs. 72.92%), where rule-based pattern recognition of missing polymorphism is effective without requiring scenario simulation. For the structurally deep principles—DIP (91.67%), ISP (97.92%), and LSP (81.25%)—Diff Eval’s diff signal makes abstract violations concretely observable, delivering large gains over both baselines. Single Agent struggles particularly on DIP (20.83%), reflecting that tight coupling is abstract until the code is forced to change.

4.3 Accuracy by Difficulty Level

Table 4.3 shows performance across three difficulty levels, defined by code length.

Difficulty	Single Agent	Two-Agent	Diff Eval
Easy (~33 LOC)	75.00%	60.00%	92.50%
Moderate (~140 LOC)	71.25%	62.50%	91.25%
Hard (~306 LOC)	63.75%	45.00%	80.00%

Table 4.3: Accuracy by difficulty level (80 examples each). Difficulty is determined by the number of lines of code.

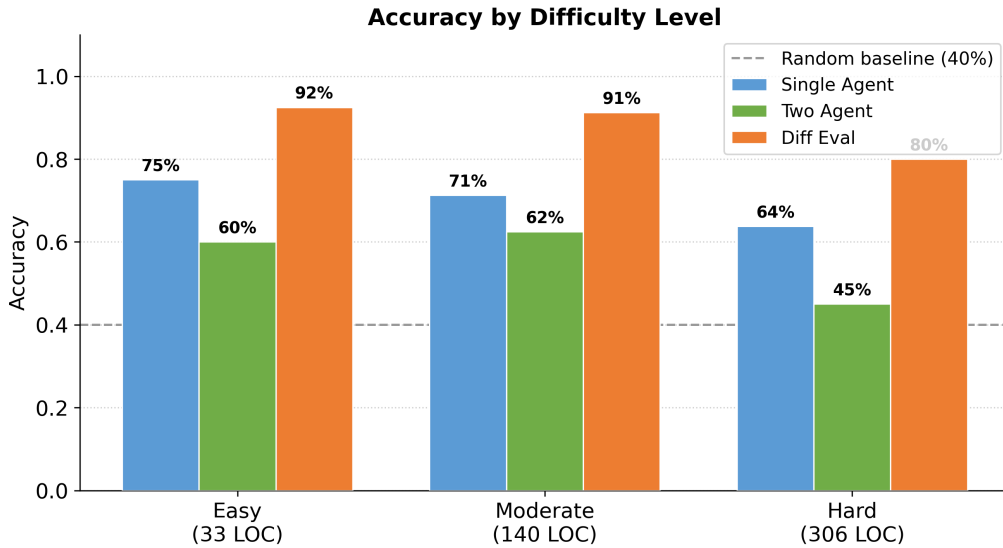


Figure 4.3: Top-2 accuracy comparison by difficulty level across the three methods.

Diff Eval leads at every difficulty level and degrades the least with increasing code length: its drop from Easy to Hard is 12.5 pp (92.50% → 80.00%), compared to 11.25 pp for Single Agent (75.00% → 63.75%) and 15 pp for Two-Agent (60.00% → 45.00%). Critically, Diff Eval

maintains 80.00% even on Hard examples, whereas the baselines fall to 63.75% and 45.00% respectively, showing a sustained accuracy advantage of 16–35 pp on the most challenging inputs.

This robustness is attributable to the Pre-Verification stage in each detection channel. The pre-verifier receives only the *diff* and the *principle-specific signal mapping*—not the full original code. This effectively compresses the context: regardless of whether the original code is 33 or 306 lines, the diff highlights only the relevant changes. The Unified Ranking node therefore operates on pre-digested, principle-specific evidence rather than raw code, insulating it from the noise introduced by increasing lines of code.

4.4 Comparison with Full-Size LLM

We evaluate **Qwen3.5 Plus** [13] (released 2026-02-15)—the state-of-the-art closed-source model from the Qwen family as of March 2026—using the identical Single Agent prompt, isolating workflow design as the sole variable.

System	Top-2 Acc.	MRR@2
Qwen3.5 9B — Single Agent (local)	70.00%	0.638
Qwen3.5 9B — Diff Eval (local)	87.92%	0.792
Qwen3.5 Plus — Single Agent (cloud)	95.83%	0.858

Table 4.4: Qwen3.5 9B workflows vs. Qwen3.5 Plus (cloud). Diff Eval closes $\sim 70\%$ of the performance gap through workflow design alone.

The plain 9B baseline lags the full-size model by 25.83 pp; Diff Eval recovers 17.92 pp of that gap without any increase in model size or cloud dependency, demonstrating that **workflow design is a primary lever for closing the capability gap between local and cloud-scale models.**

Chapter 5

DISCUSSION

5.1 Error Analysis

Figure 5.1 shows the Top-2 resolved confusion matrix for Diff Eval. The dominant off-diagonal cluster is **SRP over-prediction**: OCP→SRP (11 cases), LSP→SRP (5), and DIP→SRP (3) account for 19 of the 29 errors. Responsibility-mixing symptoms arise in many violation types—an OCP violation forcing edits across methods, or a DIP violation adding cross-cutting concerns, can resemble SRP at the diff level, suggesting the Unified Ranking step would benefit from a penalty for over-frequent SRP selection. LSP is the noisiest category (scatter to SRP and DIP), while ISP and SRP themselves are near-perfect (47/48 and 46/48).

5.2 Why Diff Eval Works

Diff Eval converts an abstract judgment (“is this code SOLID?”) into a concrete, observable task (“what changed in the diff?”). This grounds detection in objective structural evidence rather than subjective LLM assessment, applies principle-specific probing that surfaces violations under exactly the conditions where they manifest, and decomposes a complex judgment into simpler, verifiable pipeline stages.

DIP and ISP benefit most: these principles are inherently about dependencies and interfaces—

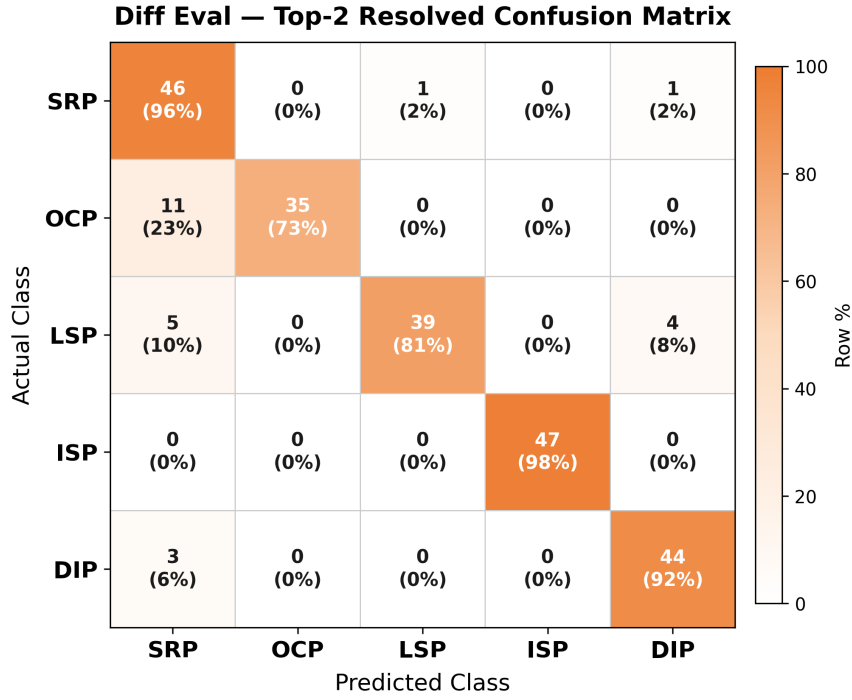


Figure 5.1: Top-2 resolved confusion matrix for Diff Eval. Dominant error: SRP over-prediction (19 off-diagonal cases).

invisible in a static snapshot but concrete when code is forced to accommodate a change. Diff Eval reaches 91.67% on DIP (*matching Qwen3.5 Plus*) and 97.92% on ISP, against 20.83% and 83.33% for Single Agent. **OCP** is the exception: its surface-visible `if-else/switch` pattern is better captured by direct inspection, explaining the 97.92% vs. 72.92% gap in favour of Single Agent. **Hard examples** favour Diff Eval (80.00% vs. 63.75% SA / 45.00% TA) because Pre-Verification receives only the compact diff, not the full source.

5.3 Privacy and Observability

Enterprise and government codebases frequently contain regulated or proprietary data that cannot be sent to external APIs without violating compliance requirements—including the EU AI Act [2], Katakri 2020 [9], and broader cloud privacy concerns documented in the literature [4]. Diff Eval runs entirely on consumer hardware (RTX 4060 8GB) via Ollama [10] with **zero data egress**, achieving 87.92% accuracy while closing 70% of the gap to cloud-scale inference.

Beyond privacy, Diff Eval is fully **observable**: every intermediate artifact—scenario description,

modified code, diff, per-principle verdicts, and ranking rationale—is persisted and inspectable. Developers can trace any verdict back to the specific diff that produced it, turning the tool from a black-box classifier into a transparent reasoning chain that supports productive disagreement rather than blind acceptance.

5.4 Limitations

- **OCP accuracy:** Diff Eval’s weakest result is OCP (72.92%). The root cause is scenario cross-contamination: the SRP extension scenario is broad enough to absorb OCP violation signals, causing SRP to rank first. This reflects a deeper ambiguity in SOLID itself—“responsibility” and “extension” are notoriously ill-defined, and their conceptual overlap is a structural property of the taxonomy. Pehlivan et al. [12] observe the same pattern: DIP violations are disproportionately misclassified as SRP or OCP, confirming the issue is not model-specific.
- **Runtime cost:** 20 inference calls per example (mean 151.6s) versus 1 call for Single Agent (2.1s). Offset by local execution and GPU-level parallelism potential.
- **Dataset scope:** The 240-example benchmark is relatively small; larger-scale evaluation would strengthen the findings.

Chapter 6

CONCLUSIONS

This work demonstrates a core principle: **Small LLM + Agentic Workflow $\approx$ Full-Size LLM, in the code quality detection scenario**. The Diff Eval approach—testing code by attempting to extend it and observing the resulting diff—achieves 87.92% accuracy on SOLID principle violation detection using a local 9B model, closing approximately 70% of the performance gap to a full-size cloud LLM (Qwen3.5 Plus, 95.83%) that uses the identical single-agent prompt. The plain small-model baseline achieves only 70.00%; Diff Eval recovers 17.92 of the 25.83 pp deficit through workflow design alone, with zero increase in model size and zero data egress.

The key insight is that design quality is better revealed through **behavioral observation (what happens when you try to change the code)** than through *static judgment* (does this code look right)—a process that mirrors how developers actually reason about code quality during real-world software development. This principle—converting abstract reasoning into concrete, observable evidence—may generalize beyond SOLID detection to other areas of software quality assessment.

Future work includes: (1) a hybrid approach combining the best method per principle, (2) improved OCP scenario generation, (3) larger-scale evaluation with diverse codebases, and (4) integration into continuous integration pipelines for real-time code review.

REFERENCES

- [1] Eman Abdullah AlOmar et al. “Behind the Intent of Extract Method Refactoring: A Systematic Literature Review”. In: *IEEE Transactions on Software Engineering* 49.4 (2023), pp. 2709–2727. DOI: [10.1109/TSE.2022.3221736](https://doi.org/10.1109/TSE.2022.3221736).
- [2] European Parliament and Council of the European Union. *Regulation (EU) 2024/1689 of the European Parliament and of the Council Laying Down Harmonised Rules on Artificial Intelligence (Artificial Intelligence Act)*. Tech. rep. L 2024/1689. Entered into force 1 August 2024. Official Journal of the European Union, 2024.
- [3] Angela Fan et al. “Large Language Models for Software Engineering: Survey and Open Problems”. In: *arXiv preprint arXiv:2310.03533* (2023).
- [4] Ahlem Ghorbel, Mahmoud Ghorbel, and Mohamed Jmaiel. “Privacy in Cloud Computing Environments: A Survey and Research Challenges”. In: *The Journal of Supercomputing* 73.6 (2017), pp. 2763–2800. DOI: [10.1007/s11227-016-1953-y](https://doi.org/10.1007/s11227-016-1953-y).
- [5] LangChain Team. *LangGraph: Building Stateful, Multi-Actor Applications with LLMs*. <https://github.com/langchain-ai/langgraph>. Accessed: 2025. 2024.
- [6] Robert C. Martin. *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, 2017.
- [7] Robert C. Martin. “Design Principles and Design Patterns”. In: Object Mentor Technical Report. 2000.
- [8] Rian Melo et al. *Agentic LMs: Hunting Down Test Smells*. 2025. arXiv: [2504.07277](https://arxiv.org/abs/2504.07277) [[cs.SE](#)].

- [9] Ministry for Foreign Affairs of Finland. *Information Security Auditing Tool for Authorities* — *Katakri 2020*. Tech. rep. National Security Authority of Finland, 2020.
- [10] Ollama Team. *Ollama: Run Large Language Models Locally*. <https://ollama.com>. Accessed: 2025. 2024.
- [11] Fabio Palomba et al. “On the diffuseness and the impact on maintainability of code smells: a large scale empirical investigation”. In: *Proceedings of the 40th International Conference on Software Engineering (ICSE)*. 2018, pp. 1188–1198. DOI: [10.1145/3180155.3180220](https://doi.org/10.1145/3180155.3180220).
- [12] Fatih Pehlivan et al. “Are We SOLID Yet? An Empirical Study on Prompting LLMs to Detect Design Principle Violations”. In: *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*. Bilkent University, Turkey. 2025.
- [13] Qwen Team. *Qwen3.5: Accelerating Productivity with Native Multimodal Agents*. <https://qwen.ai/blog?id=qwen3.5>. Feb. 2026.
- [14] Tushar Sharma and Diomidis Spinellis. “A survey on software smells”. In: *Journal of Systems and Software* 138 (2018), pp. 158–173. DOI: [10.1016/j.jss.2017.12.034](https://doi.org/10.1016/j.jss.2017.12.034).
- [15] Lei Wang et al. “A Survey on Large Language Model based Autonomous Agents”. In: *Frontiers of Computer Science* 18.6 (2024). DOI: [10.1007/s11704-024-40231-1](https://doi.org/10.1007/s11704-024-40231-1).
- [16] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *arXiv preprint arXiv:2210.03629* (2023).